Guia de consulta para a apresentação

Mapa de cada módulo/classe do projeto: o que faz e por que importa. Detalhes finos estão em motor-pix.md, providers-pix.md, storage-pix.md e api-pix.md.

Visão em uma frase

API (FastAPI + PostgreSQL) que recebe uma transação Pix, enriquece com dados do banco e de reputação (DICT), roda um **motor de regras determinístico** e devolve `APPROVED`, `TWO_FA` ou `REJECTED` com score e motivos. Cada análise é imutável, auditável e idempotente.

Fluxo de um `POST /pix/transaction`

1. **Middlewares**: gera request id ç loga ç valida `INTERNAL-TOKEN`, resolve o tenant e abre o orçamento de tempo (3 s) ç abre sessão de banco.
2. **PixResource** lê o corpo bruto (limite 64 KiB) e registra um recibo.
3. **PixController** normaliza o JSON, reserva a transação (claim), enriquece, persiste e decide.
4. **Enricher** consulta banco e DICT (com timeout, 1 retry e cache) e monta o `Context`.
5. **Motor** (`evaluate`) calcula o score e o status.
6. **PixStorage/PixRepository** gravam tudo atomicamente; a resposta pública sai em 201 (ou 200 se for repetição).

---

1. Motor antifraude — `src/antifraud/` (o coração do projeto)

Módulo / classe	Função	Por que importa
`contract.py` — `parse_payload_json`, `normalize_payload`, `NormalizedPayload`, `ValidationError`	Lê o JSON rejeitando chaves duplicadas, NaN/Infinity e texto inválido para JSONB; normaliza CPF/CNPJ e datas para UTC; gera o `payload_hash`.	Entrada do sistema. O hash permite detectar o mesmo `id` reenviado com corpo diferente (409). O erro carrega código e HTTP prontos.
`adapter.py` — `to_pix`	Converte o payload normalizado no objeto de domínio `Pix`.	Isola o formato HTTP do domínio.
`context.py` — `ImmutableValue`, `Pix`, `AccountKey`, `DestinationKey`, `ExecutionEvent`, `AuthEvent`, `Restriction`, `SecuritySnapshot`, `DictEvidence`, `HistoricalAnalysis`, `BankPayment`, `ProviderTrace`, `Context`, `Reason`, `Decision`	Modelos de domínio imutáveis e validados na criação. `Context` é tudo que o motor enxerga: a transação, o histórico, o estado do dispositivo, restrições e evidência do DICT. `Reason` é um motivo (código, família, pontos, piso). `Decision` é o resultado (status, score, famílias, motivos, `c_pre`).	Imutabilidade garante determinismo e auditoria: mesma entrada, mesma decisão. Tudo que a regra usa fica explícito.

Módulo / classe	Função	Por que importa
`policy.py` — `Policy`	Parâmetros da política (cortes 30/70, tetos por família, janelas, limites em centavos, horário noturno etc.), validados no construtor. Tem `fingerprint` e `configuration`.	Toda constante de negócio vive aqui, não espalhada em regras. O fingerprint prova qual política produziu cada decisão.
`configuration.py` — `PolicyCatalog`, `load_policy_catalog`	Carrega `policies.json` e valida com JSON Schema; `for_tenant` entrega a política de cada instituição.	Cada tenant pode ter política própria sem mudar código. Rejeita configuração inválida na subida.
`policies.json`	Política `pix-experimental-v3`.	Pesos e cortes são escolhas experimentais do grupo, não calibração de fraude. Dizer isso explicitamente na apresentação.
`features.py` — `Features`, `build_features`	Deriva sinais do `Context`: mediana do histórico, soma diária, fan-in de pagadores, janela noturna, exposição por dispositivo. Cálculos pesados são lazy.	Separa "medir" de "julgar": as regras só leem números prontos.
`rules.py` — `Criterion`, `CRITERIA`	25 critérios puros (ex.: `VELOCITY_BURST`, `STRUCTURING`, `MULE_FAN_IN`, `DEVICE_CHANGE`, `PAYEE_DICT_ADVERSE`) em 6 famílias: Limite, Velocity, Montante, Perfil, Acesso, Contraparte. Cada um devolve pontos e, às vezes, um piso (força 2FA ou rejeição). Fase `local` ou `reputation`.	É a lógica de negócio. Pisos de bloqueio ficam no código e não podem ser desligados pelo JSON do tenant. A ordem é fixa, o que facilita auditoria.
`engine.py` — `evaluate`, `preliminary`	Soma pontos por família com teto, aplica cortes (≥30 = 2FA, ≥70 = rejeita), aplica pisos e calcula `c_pre` (confiança antes do DICT, descontando incerteza). `preliminary` roda sem reputação.	Decisão final e determinística. `preliminary` decide se vale a pena consultar o DICT.
`dto.py` — `PixTransactionDTO`	Monta a resposta `public` (`key`, `status`, `metadata`) e o `audit` completo.	Separa o que o cliente vê do que fica guardado para auditoria.

2. Providers — `src/providers/` (dados externos)

Módulo / classe	Função	Por que importa
`contracts.py` — `BankContextProvider`, `ReputationProvider` (Protocols); `BankRecord`, `ReputationRecord`, `BankLink`, `ProviderIdentity`, `SecurityScope`, `ReputationKey`; erros `ProviderDataError`, `ReconciliationRequired`, `ProviderUnavailable`	Contrato das fontes: banco (histórico, dispositivo, restrições) e reputação (DICT). Valida versão, escopo e formato dos dados.	Define o que é dado confiável. Dado inválido nunca é tratado como "limpo".

Módulo / classe	Função	Por que importa
`fixtures.py` — `FixtureBankProvider`, `FixtureReputationProvider`; `configuration.py` — `load_fixture_providers`	Implementações de teste lidas de arquivo.	Permitem demo e testes reproduzíveis. **Não comprovam integração com DICT real.**
`budget.py` — `Deadline`, `ProviderRunner`, `BudgetExceeded`	Orçamento de tempo único por requisição; cada chamada tem timeout (0,25 s) e no máximo 1 retry técnico.	Garante resposta dentro do prazo. Se estourar, não decide com resultado tardio: devolve 503.
`selection.py` — `select_security`, `select_reputation`, `reconcile_payments`	Escolhe o snapshot válido mais recente (TTL de 5 min para DICT) e reconcilia pagamentos com vínculos do banco.	Evita usar dado velho ou conflitante.
`enrichment.py` — `Enricher`, `Enrichment`, `EnrichmentProgress`, `EnrichmentNeedsRefresh`, `SourceCheck`	Orquestra as consultas: calcula `preliminary`, consulta o DICT só quando necessário, usa cache e monta o `Context` final. Se o contexto mudar no meio, sinaliza refresh e preserva o progresso.	Une provedores e motor. Faz DICT condicional, o que economiza chamadas e tempo.

3. Camada HTTP

Módulo / classe	Função	Por que importa
`app.py` — `create_app`	Carrega políticas, tenants e provedores, registra middlewares, rotas e handlers.	Ponto de montagem. Falha ao subir se a configuração for inválida.
`resources/pix.py` — `PixResource`	`on_post` e `on_get`; lê o corpo com limite de 64 KiB e timeout; converte falhas em `PixError`. O FastAPI nunca decodifica o JSON financeiro.	Fronteira HTTP fina. Evita que o framework altere valores financeiros.
`controllers/pix_api.py` — `PixController`	Orquestra o POST (normalizar ç claim ç enriquecer ç persistir ç decidir, até 3 ciclos de revalidação) e o GET. Mapeia exceções para o código de erro certo.	Regra de fluxo central: idempotência, replay (200), conflito (409), orçamento (503).
`controllers/pix_storage.py` — `PixStorage`, `AnalysisWork`	Transações de banco: claim com lease e fencing token, locks com timeout, recibos, vínculos de desafio/operação bancária, ingestão de eventos.	Garante que duas requisições concorrentes com o mesmo `id` não gerem duas análises, e que um worker antigo não grave por cima do novo.
`repositories/pix_repository.py` — `PixRepository`	Todo o SQL: claims, contas, análises, snapshots, eventos, recibos; monta o `Context` a partir do histórico persistido.	Único ponto de acesso ao banco do Pix.

Módulo / classe	Função	Por que importa
`models/pix.py`	Tabelas SQLAlchemy Core (`pix_transaction`, `risk_analysis`, `analysis_claim`, `request_receipt`, `bank_event`, `provider_snapshot` etc.). DDL vem das migrations em `migrations/`.	Esquema único; análise imutável com `snapshot` e `public_response` guardados.
`dtos/pix_storage.py`	Tipos de resultado do storage (`Replay`, `Busy`, `HashConflict`, `OwnershipConflict`).	Torna explícitos os desfechos de concorrência.
`database.py`	Engine do Pix e `Context`/sessão por requisição.	Uma sessão por requisição, limpa no fim.
`utils/pix_deadline.py` — `in_database`, `DatabaseClock`	Executa operações de banco em thread respeitando o `Deadline`; relógio ancorado no PostgreSQL.	Commit tardio nunca vira 201. Relógio local não decide leases.
`utils/pix_tenants.py` — `Tenant`, `load_tenants`, `resolve_tenant`	Tenant (nome, token, ISPB) definido no servidor e resolvido pela credencial com comparação em tempo constante.	O cliente nunca informa o tenant, o que evita falsificação.
`utils/pix_storage_codec.py`, `utils/document_number.py`	Serialização do snapshot e normalização de CPF/CNPJ.	Consistência dos dados gravados.

4. Middlewares (ordem de execução)

1. `request_context` — gera o `request_id`.
2. `request_logger` — loga entrada e saída.
3. `internal_token` — exige `INTERNAL-TOKEN`; nas rotas Pix resolve o tenant e define o orçamento de 3 s.
4. `session_manager` — abre e fecha a sessão de banco.

5. Erros — `src/errors/`

- `base_error.py` / `custom_errors.py` — `QIException` e erros padrão do Base Service.
- `pix.py` — `PixError`: cada código `QITxxxxxx` tem um HTTP fixo (ex.: 404 `QIT002001`, 409 `QIT002002`, 413 `QIT002009`, 503 `QIT002008`). Combinação inconsistente levanta erro.
- `handlers.py` — transforma exceções em resposta JSON padronizada.

6. Herdado do Base Service (template, fora do escopo Pix)

`sample_entity` (resource, controller, repository, DTO, models), `bankslip_connector.py`, `rest_connector.py` (cliente HTTP com timeout obrigatório e logs), `health_check.py`, `utils/logger.py`, `schema_handler.py`. Servem de padrão de estrutura e de exemplo; o Pix é o que foi implementado de novo.

Pontos fortes para destacar

- **Determinismo e auditoria**: contexto imutável, política versionada com fingerprint, motivos explícitos.
- **Idempotência real**: claim com lease e fencing, hash do payload, replay devolve 200.
- **Pisos de segurança em código**: nenhum tenant desliga um bloqueio via JSON.
- **DICT condicional**: só consulta quando o resultado preliminar justifica.
- **Orçamento de tempo ponta a ponta**: nada decide com resposta atrasada.
- **Multi-tenant** por credencial e política por instituição.

Ressalvas honestas

- Score não é probabilidade de fraude; pesos e cortes são experimentais.
- Provedores são fixtures; não há integração real com DICT.
- Execução do Pix, frontend e 2FA ficam fora do escopo.